

الگوریتم‌های پرتکرار در امتحان IHK

در این فایل چند نمونه از الگوریتم‌های معمول در آزمون‌های حرفه‌ای IHK (بخش توسعه و پیاده‌سازی الگوریتم‌ها) گردآوری شده است. برای هر موضوع، یک شبه‌کد به سبک استاندارد IHK و نسخه معادل آن به سبک شبه‌جاوا ارائه می‌شود و سپس توضیحی کوتاه به زبان فارسی درباره منطق حل مسئله بیان شده است.

مرتب‌سازی آرایه‌ها

شبه‌کد به سبک IHK

```
Algorithmus sortiereArray(A: Array von Integer):  
    N = A.length  
    für i = 0 bis N-2:  
        minIndex = i  
        für j = i+1 bis N-1:  
            wenn A[j] < A[minIndex] dann  
                minIndex = j  
        temp = A[i]  
        A[i] = A[minIndex]  
        A[minIndex] = temp
```

شبه‌کد معادل (شبه‌جاوا)

```
void sortiereArray(int[] A) {  
    int N = A.length;  
    for (int i = 0; i < N - 1; i++) {  
        int minIndex = i;  
        for (int j = i + 1; j < N; j++) {  
            if (A[j] < A[minIndex]) {  
                minIndex = j;  
            }  
        }  
        // swap elements  
        int temp = A[i];  
        A[i] = A[minIndex];  
        A[minIndex] = temp;  
    }  
}
```

توضیحات

الگوریتم فوق نمونه‌ای از مرتب‌سازی انتخابی (Selection Sort) است که در آن برای هر موقعیت، کوچک‌ترین عنصر باقیمانده در آرایه پیدا شده و جای آن با عنصر در موقعیت جاری عوض می‌شود. در امتحان، زمانی که از شما خواسته می‌شود داده‌ها را به ترتیب صعودی یا نزولی مرتب کنید، یکی از روش‌های قابل قبول استفاده از همین الگوریتم‌های ساده است. مهم‌ترین نکته، استفاده صحیح از حلقه‌های تو در تو و کنترل محدوده اندیس‌هاست. زمان اجرای این الگوریتم تقریباً برابر با $O(n^2)$ است؛ با این حال برای آرایه‌های کوچک در آزمون مناسب است.

آرایه‌های دوبعدی و پردازش داده‌ها

شبکه‌کد به سبک IHK

```
Algorithmus findeBesucherreichstenTag(matrix: Array[0..T-1][0..H-1] von
Integer): Integer
    maxSumme = -∞
    tagIndex = -1
    für t = 0 bis T-1:
        summe = 0
        برای h = 0 bis H-1:
            summe = summe + matrix[t][h]
        اگر summe > maxSumme آنگاه
            maxSumme = summe
            tagIndex = t
    zurückgeben tagIndex
```

شبکه‌کد معادل (شبکه‌جاوا)

```
int findeBesucherreichstenTag(int[][] matrix) {
    int T = matrix.length;
    int H = matrix[0].length;
    int maxSumme = Integer.MIN_VALUE;
    int tagIndex = -1;
    for (int t = 0; t < T; t++) {
        int summe = 0;
        for (int h = 0; h < H; h++) {
            summe += matrix[t][h];
        }
        if (summe > maxSumme) {
            maxSumme = summe;
            tagIndex = t;
        }
    }
}
```

```
return tagIndex;  
}
```

توضیحات

این مثال ساده کار با آرایه‌های دوبعدی را نشان می‌دهد. فرض کنید `matrix` نمایانگر جدول تعداد بازدیدکنندگان در ساعات مختلف برای روزهای مختلف است؛ سپس می‌خواهیم روزی را که مجموع بازدید بیشترین مقدار را دارد بیابیم. برای این کار، با دو حلقه تو در تو (یک حلقه برای سطرها و دیگری برای ستون‌ها) روی تمامی عناصر ماتریس پیمایش می‌کنیم و مجموع هر سطر را می‌یابیم. بیشترین مقدار مجموع در متغیر `maxSumme` نگهداری می‌شود و اندیس آن روز در `tagIndex`. این الگو در بسیاری از مسائل امتحانی که شامل جمع یا بررسی داده‌ها در آرایه‌های دوبعدی هستند، کاربرد دارد.

تابع مقایسه اشیا

شبه‌کد به سبک IHK

```
Algorithmus vergleicheMitarbeiter(m1: Mitarbeiter, m2: Mitarbeiter): Integer  
    diff = m1.getGehalt() - m2.getGehalt()  
    wenn diff > 0 dann  
        Rückgabe 1  
    sonst wenn diff < 0 dann  
        Rückgabe -1  
    sonst  
        Rückgabe 0
```

شبه‌کد معادل (شبه‌جاوا)

```
int vergleicheMitarbeiter(Mitarbeiter m1, Mitarbeiter m2) {  
    double diff = m1.getGehalt() - m2.getGehalt();  
    if (diff > 0) {  
        return 1;  
    } else if (diff < 0) {  
        return -1;  
    } else {  
        return 0;  
    }  
}
```

توضیحات

در برخی مسائل نیاز است دو شیء را بر اساس یک ویژگی مقایسه کنیم و نتیجه این مقایسه را در قالب عددی مثبت، صفر یا منفی بازگردانیم. توابعی از این نوع به نام *تابع مقایسه* (*Comparator*) شناخته می‌شوند. در مثال بالا دو شیء از نوع *Mitarbeiter* بر اساس حقوقشان مقایسه می‌شوند. اگر حقوق نخستین بیشتر باشد، مقدار مثبت (مثلاً 1) بازگردانده می‌شود؛ اگر کمتر باشد، مقدار منفی؛ و اگر برابر باشد، صفر. این ساختار برای مرتب‌سازی مجموعه‌ای از اشیا با استفاده از تابع مقایسه و نیز برای انعطاف‌پذیرتر کردن الگوریتم‌های مرتب‌سازی به کار می‌رود.

تبدیل عدد دهدهی به دودویی

شبه‌کد به سبک IHK

```
Algorithmus dezimalZuBinaer(n: Integer): String
    wenn n = 0 آنگاه
        بازگردان "0"
    ergebnis = ""
    n > 0: مادامی که
        rest = n mod 2
        ergebnis = konvertiereZuZeichen(rest) + ergebnis
        n = n / 2 // integer division
    بازگردان ergebnis
```

شبه‌کد معادل (شبه‌جاوا)

```
String dezimalZuBinaer(int n) {
    if (n == 0) return "0";
    String result = "";
    while (n > 0) {
        int bit = n % 2;
        result = bit + result;
        n = n / 2;
    }
    return result;
}
```

توضیحات

برای تبدیل یک عدد دهدهی به نمایش دودویی (باینری)، از تقسیم متوالی عدد بر ۲ استفاده می‌کنیم و باقی‌مانده‌های ۰ یا ۱ حاصل را به ترتیب از کم ارزش‌ترین رقم به پرارزش‌ترین کنار هم می‌گذاریم. در این الگوریتم، تا زمانی که n بزرگ‌تر از صفر باشد، عدد را بر ۲ تقسیم می‌کنیم و باقی‌مانده را در ابتدای رشته

نتیجه قرار می‌دهیم. این روش ساده و رایج است و در امتحان نیز معمولاً به همین صورت انتظار می‌رود. اگر ورودی صفر باشد، خروجی مستقیماً رشته «» خواهد بود.

فشرده‌سازی داده‌های تکراری

شبیه‌کد به سبک IHK

```
Algorithmus erstelleKomprimierung(eingabe: Array von String): Array von String
    wenn eingabe.length = 0 آنگاه
        {} بازگردان
    ergebnisListe = لیست خالی
    aktuelles = eingabe[0]
    zaehler = 1
    برای i = 1 تا eingabe.length - 1:
        آنگاه eingabe[i] = aktuelles
            zaehler = zaehler + 1
        sonst
            ergebnisListe.افزودن(aktuelles + zaehler)
            aktuelles = eingabe[i]
            zaehler = 1
    ergebnisListe.افزودن(aktuelles + zaehler)
    Array به صورت ergebnisListe بازگردان
```

شبیه‌کد معادل (شبیه‌جاوا)

```
String[] erstelleKomprimierung(String[] eingabe) {
    if (eingabe.length == 0) {
        return new String[0];
    }
    List ergebnisListe = new ArrayList<>();
    String aktuelles = eingabe[0];
    int zaehler = 1;
    for (int i = 1; i < eingabe.length; i++) {
        if (eingabe[i].equals(aktuelles)) {
            zaehler++;
        } else {
            ergebnisListe.add(aktuelles + zaehler);
            aktuelles = eingabe[i];
            zaehler = 1;
        }
    }
    ergebnisListe.add(aktuelles + zaehler);
    return ergebnisListe.toArray(new String[0]);
}
```

}

توضیحات

الگوریتم فوق یک روش ساده برای فشرده‌سازی توالی‌های تکراری (Run-Length Encoding) است. در آرایه ورودی، کاراکترها یا رشته‌های پشت سر هم را شمارش می‌کنیم و نتیجه را به صورت ترکیب مقدار و تعداد تکرار آن ذخیره می‌کنیم؛ مثلاً اگر ورودی ["A", "A", "B", "B", "B"] باشد، خروجی ["A2", "B3"] خواهد بود. این روش در سوالات امتحانی که به کاهش حجم داده‌های تکراری اشاره می‌کنند بسیار کاربردی است. لازم است ابتدا آرایه را از ابتدا تا انتها پیمایش کنید؛ زمانی که مقدار فعلی تغییر کرد، نتیجه قبلی را به لیست خروجی اضافه کنید و شمارش را برای مقدار جدید از ابتدا شروع کنید. در پایان نیز آخرین بخش فشرده‌شده را اضافه کنید.

مرتب‌سازی لیست جنریک با تابع مقایسه

شبه‌کد به سبک IHK

```
Algorithmus sortiereListe(liste: List<T>, comp: Funktion(T, T) → Integer): void
    برای i = 0 تا liste.size() - 2:
        برای j = 0 تا liste.size() - 2 - i:
            آنگاه wenn comp(liste[j], liste[j+1]) > 0
                tausche liste[j] und liste[j+1]
```

شبه‌کد معادل (شبه‌جاوا)

```
<T> void sortList(List<T> liste, Comparator<T> comp) {
    for (int i = 0; i < liste.size() - 1; i++) {
        for (int j = 0; j < liste.size() - 1 - i; j++) {
            if (comp.compare(liste.get(j), liste.get(j+1)) > 0) {
                T tmp = liste.get(j);
                liste.set(j, liste.get(j+1));
                liste.set(j+1, tmp);
            }
        }
    }
}
```

توضیحات

در مسائلی که نیاز است لیستی از اشیا با هر نوع دلخواه (T) مرتب شود، می‌توان تابع مرتب‌سازی را به صورت جنریک نوشت و یک تابع مقایسه جداگانه به آن ارسال کرد. این روش انعطاف‌پذیری بالایی ایجاد می‌کند زیرا با تغییر تابع مقایسه می‌توان معیار مرتب‌سازی را تغییر داد. در شبه‌کد بالا، از نسخه ساده مرتب‌سازی حبابی (Bubble Sort) استفاده شده است که بر اساس نتیجه تابع مقایسه comp تصمیم می‌گیرد دو عنصر جا به جا شوند یا خیر. این ساختار در سوالات مربوط به «مرتب‌سازی با استفاده از تابع مقایسه» یا «استفاده از Generics» به کار می‌رود.

شمارش بازدیدکنندگان

شبه‌کد به سبک IHK

```
Algorithmus countVisitors(start: EventNode): Integer
    count = 0
    current = start
    solange current ≠ null:
        wenn current.typ = "COME" آنگاه
            count = count + 1
        sonst // assume LEAVE
            count = count - 1
        current = current.naechstes
    بازگردان count
```

شبه‌کد معادل (شبه‌جاوا)

```
int countVisitors(EventNode start) {
    int count = 0;
    EventNode current = start;
    while (current != null) {
        if (current.getType().equals("COME")) {
            count++;
        } else {
            count--;
        }
        current = current.getNext();
    }
    return count;
}
```

توضیحات

این الگوریتم نمونه‌ای از حل مسائل مربوط به شمارش رویدادها در داده‌های پیوندی یا سلسله‌وار است. با یک متغیر شمارنده و پیمایش لیست رویدادها، هر بار که شخصی وارد می‌شود (COME) یک واحد به شمارنده اضافه می‌شود و هر بار که خارج می‌شود (LEAVE) یک واحد کم می‌شود. در انتها مقدار شمارنده، تعداد افراد حاضر را نشان می‌دهد. این ساختار در سوالاتی به کار می‌رود که داده‌ها به صورت دنباله‌ای از رویدادها در یک ساختار لینک‌شده ذخیره شده‌اند.

این فایل به عنوان مروری بر چند الگوریتم پایه‌ای و پرکاربرد در آزمون‌های توسعه الگوریتم‌های HK تهیه شده است. حفظ الگوهای کلی و درک منطق پشت آنها می‌تواند در حل سوالات مشابه در آینده بسیار مفید باشد.